

MEMORIA TÉCNICA

Proyecto de Machine Learning aplicado al deporte

Predicción de Trayectoria en Deportes

*Framework de clasificación multiclase bajo índice
de presión ponderado*

Repositorio: `github.com/jairosdc/
Prediccion-de-trayectoria-en-deportes`

Tecnologías: Python 3.10+, scikit-learn, Streamlit, pandas

Versión: main (última revisión)

Tipo: Ingeniería inversa completa

Abstract

Este documento constituye la memoria técnica completa del proyecto *Predicción de Trayectoria en Deportes*, obtenida mediante ingeniería inversa del código fuente disponible en el repositorio público. El proyecto implementa un framework de *machine learning* para la predicción de la zona de decisión más probable en situaciones deportivas de alta presión (duelos 1 contra 1), cubriendo seis disciplinas: fútbol, tenis, baloncesto, balonmano, hockey sobre hielo y eSports.

La arquitectura consta de cuatro componentes principales: (1) un generador de datos sintéticos basado en distribuciones de probabilidad controladas, (2) un índice de presión ponderado multifactor $P = 0,30T + 0,25S + 0,20C + 0,15F + 0,10M$, (3) un pipeline de clasificación logística multinomial con codificación *one-hot* y estandarización, y (4) una interfaz web interactiva desarrollada en Streamlit.

El modelo opera sobre 1.800 eventos sintéticos (300 por deporte), 13 variables predictoras y 22 clases de salida, alcanzando una exactitud global de aproximadamente el 34%, valor que contrasta favorablemente con la referencia aleatoria del 4,5% para 22 clases equiprobables.

Palabras clave: clasificación multiclase, regresión logística, índice de presión, datos sintéticos, análisis deportivo, scikit-learn, Streamlit.

Contents

1	Introducción y motivación	4
1.1	Contexto del proyecto	4
1.2	Objetivos	4
1.3	Alcance	4
2	Arquitectura y estructura del repositorio	5
2.1	Organización de archivos	5
2.2	Flujo de datos (pipeline completo)	5
2.3	Dependencias	6
3	Generación de datos sintéticos	7
3.1	Justificación	7
3.2	Estructura del dataset	7
3.3	Distribuciones de probabilidad por variable	8
3.3.1	Variables numéricas de contexto	8
3.3.2	Variable de lateralidad	9
3.3.3	Variable objetivo: <code>decision_zone</code>	9
3.4	Zonas de decisión por deporte	10
3.5	Reglas de comportamiento programadas	10
3.6	Semillas de reproducibilidad	12
4	Índice de presión ponderado	13
4.1	Definición formal	13
4.2	Componentes	13
4.2.1	Presión temporal T	13
4.2.2	Presión por marcador S	13
4.2.3	Importancia competitiva C , fatiga F y momento crítico M	13
4.3	Pesos y justificación	14
4.4	Clasificación del índice	14
4.5	Implementación	14
5	Preprocesado de datos	16
5.1	Pipeline de preprocesado	16
5.2	Columnas obligatorias	16
5.3	Limpieza de datos	17
5.4	Variables del modelo	17

6	Modelo de Machine Learning	18
6.1	Arquitectura del pipeline de sklearn	18
6.2	Formulación matemática	18
6.2.1	Codificación y espacio de características	18
6.2.2	Regresión logística multinomial (softmax)	18
6.2.3	Función de pérdida	19
6.2.4	Decisión	19
6.3	División train/test	19
6.4	Hiperparámetros	19
7	Análisis descriptivo	20
7.1	Módulo <code>src/analysis.py</code>	20
7.2	Presión media esperada por deporte	20
8	Interfaz web (Streamlit)	21
8.1	Arquitectura de <code>app.py</code>	21
8.2	Componentes de la interfaz	21
8.3	Módulo de animaciones (<code>src/animation.py</code>)	21
9	Evaluación y resultados	23
9.1	Métricas implementadas	23
9.2	Resultados con datos sintéticos	23
9.3	Comportamiento por deporte	24
10	Análisis crítico: limitaciones y trabajo futuro	25
10.1	Limitaciones identificadas	25
10.2	Trabajo futuro	26
11	Suite de tests	27
12	Evolución del proyecto: de <code>Mates.py</code> al framework multideporte	28
12.1	Versión original: <code>Mates.py</code>	28
12.2	Generalización multideporte	28
13	Conclusiones	29
	References	30
A	Ejemplo de evento sintético completo	31
B	Ejecución del sistema	31

1 Introducción y motivación

1.1 Contexto del proyecto

La predicción del comportamiento humano en situaciones de alta presión constituye uno de los problemas más ricos en la intersección entre la ciencia cognitiva, la estadística y el aprendizaje automático. En el ámbito deportivo, este problema se concreta en la pregunta: *dado un conjunto de condiciones contextuales (momento del partido, marcador, nivel de fatiga, importancia de la competición), ¿qué decisión táctica es más probable que tome un jugador?*

El proyecto aborda esta pregunta con un enfoque modular y reproducible. En lugar de limitar el análisis a un único deporte, el sistema unifica la estructura del problema para seis disciplinas bajo un marco matemático común, permitiendo que el mismo pipeline de preprocesado, modelado y evaluación opere sobre todos ellos sin modificaciones estructurales.

1.2 Objetivos

- O1.** Definir un índice de presión contextual $P \in [0, 1]$ que capture la tensión situacional de un evento deportivo mediante la combinación ponderada de cinco factores.
- O2.** Generar un conjunto de datos sintéticos con patrones estadísticamente coherentes que permita validar la arquitectura antes de integrar datos reales.
- O3.** Entrenar un modelo de clasificación multiclase que, dado el contexto de un evento, devuelva una distribución de probabilidades sobre las posibles zonas de decisión.
- O4.** Desarrollar una interfaz web interactiva que permita explorar el modelo en tiempo real, configurando el escenario deportivo y comparando la predicción del modelo con la intuición del público.

1.3 Alcance

La versión actual del proyecto trabaja exclusivamente con datos sintéticos. Los patrones aprendidos por el modelo son, en consecuencia, los patrones que fueron programados en el generador de datos. Esto es deliberado: el objetivo de esta fase es validar que la arquitectura completa (generación $\rightarrow$ presión $\rightarrow$ preprocesado $\rightarrow$ modelo $\rightarrow$ predicción $\rightarrow$ interfaz) funciona correctamente antes de conectar fuentes de datos reales.

2 Arquitectura y estructura del repositorio

2.1 Organización de archivos

Table 1: Estructura completa del repositorio y función de cada componente.

Archivo / Directorio	Función
<code>synthetic_multisport_data.py</code>	Generador de datos sintéticos para los 6 deportes
<code>main.py</code>	Pipeline completo por línea de comandos
<code>app.py</code>	Interfaz web Streamlit
<code>Mates.py</code>	Versión original (solo fútbol, punto de partida)
<code>requirements.txt</code>	Dependencias del proyecto
<code>src/pressure.py</code>	Cálculo del índice de presión P
<code>src/preprocessing.py</code>	Carga, validación y limpieza de datos
<code>src/train_model.py</code>	Pipeline de ML (sklearn)
<code>src/analysis.py</code>	Análisis descriptivo (tablas cruzadas)
<code>src/animation.py</code>	Módulo de animaciones HTML/CSS por deporte
<code>src/football_animation.html</code>	Plantilla de animación del penalti
<code>data/</code>	Directorio con los 7 CSV generados
<code>docs/mathematical_framework.md</code>	Marco matemático formal
<code>docs/technical_report.md</code>	Informe técnico del proyecto
<code>test_synthetic_multisport_data.py</code>	Suite de tests unitarios
<code>app_corregida/</code>	Versión corregida del proyecto completo

2.2 Flujo de datos (pipeline completo)

La Figura 1 muestra el flujo completo desde la generación de datos hasta la predicción.

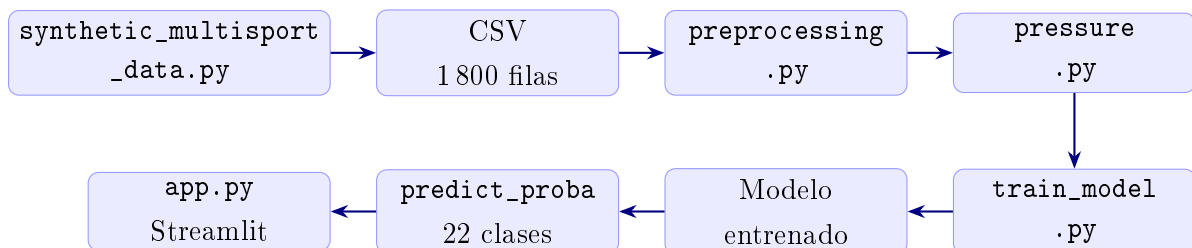


Figure 1: Pipeline completo del sistema de predicción.

2.3 Dependencias

Table 2: Dependencias principales del proyecto.

Paquete	Versión mínima	Uso
pandas	≥ 2.0	Manipulación de datos tabulares
numpy	≥ 1.24	Operaciones numéricas y muestreo
scikit-learn	≥ 1.3	Pipeline de ML (OHE, escalado, LR)
streamlit	≥ 1.30	Interfaz web
plotly	≥ 5.0	Visualizaciones interactivas

3 Generación de datos sintéticos

3.1 Justificación

Los datos reales de eventos deportivos detallados (localización exacta del tiro, contexto del partido, nivel de fatiga del jugador) son escasos, de acceso restringido y sujetos a derechos de propiedad intelectual. La opción de datos sintéticos permite:

- Validar el pipeline completo sin dependencias externas.
- Controlar exactamente los patrones estadísticos que el modelo debe aprender.
- Garantizar reproducibilidad total mediante semillas fijas.
- Facilitar la depuración de errores al conocer el comportamiento esperado.

Advertencia del propio código

El módulo `synthetic_multisport_data.py` incluye explícitamente el siguiente aviso: *“Estos datos son sintéticos. Sirven para validar el pipeline, la arquitectura y la demo interactiva. No sirven para extraer conclusiones empíricas reales sobre comportamiento deportivo.”*

3.2 Estructura del dataset

Cada evento queda representado por el vector de 16 columnas descrito en la Tabla 3.

Table 3: Columnas de cada evento en el dataset.

Columna	Tipo	Dominio	Rol
event_id	String	e.g. FTB_0001	Identificador
sport	Categórica	6 valores	Predictora
action_type	Categórica	6 valores	Predictora
competition	Categórica	Variable	Predictora
match_time	Numérica	$[0, t_{\max}]$	Predictora
score_difference	Numérica	$[-5, 5]$	Predictora
is_home_player	Binaria	$\{0, 1\}$	Predictora
player	Categórica	~ 20 por deporte	Predictora
opponent	Categórica	~ 10 por deporte	Predictora
dominant_side	Categórica	$\{\text{right}, \text{left}\}$	Predictora
fatigue_level	Numérica	$[0, 1]$	Predictora
competition_importance	Numérica	$[0, 1]$	Predictora
pressure_index	Numérica	$[0, 1]$	Calculada
pressure_level	Categórica	$\{\text{baja}, \text{media}, \text{alta}\}$	Calculada
decision_zone	Categórica	Específica/deporte	Objetivo
action_result	Categórica	Específica/deporte	Informativa

3.3 Distribuciones de probabilidad por variable

3.3.1 Variables numéricas de contexto

Para cada evento, las variables de contexto siguen las distribuciones detalladas en la Tabla 4.

Table 4: Distribuciones de las variables de contexto por deporte.

Variable	Distribución	Parámetros (fútbol)	Parámetros (hockey)
match_time	$\mathcal{U}(0, t_{\max})$	$t_{\max} = 120$ min	$t_{\max} = 65$ (fijo, OT)
score_difference	$\text{DU}(a, b)$	$\text{randint}(-2, 3)$	0 (siempre empate)
competition_importance	$\mathcal{U}(a, b)$	$\mathcal{U}(0,3, 1,0)$	$\mathcal{U}(0,4, 1,0)$
fatigue_level	$\mathcal{U}(a, b)$	$\mathcal{U}(0,1, 0,9)$	$\mathcal{U}(0,5, 1,0)$
critical_moment	$\mathcal{U}$ condicional	$\mathcal{U}(0,5, 1)$ si tensión	$\mathcal{U}(0,7, 1,0)$ (siempre)

Donde $\mathcal{U}(a, b)$ denota la distribución uniforme continua en $[a, b]$ y $\text{DU}(a, b)$ la uniforme discreta (equivalente a `numpy.random.randint`).

La variable `critical_moment` se genera de forma condicional:

$$M \sim \begin{cases} \mathcal{U}(0,5, 1,0) & \text{si el contexto indica alta tensión} \\ \mathcal{U}(0, 0,5) & \text{en caso contrario} \end{cases} \quad (1)$$

3.3.2 Variable de lateralidad

La dominancia de brazo/pierna sigue una distribución de Bernoulli:

$$\text{dominant_side} \sim \text{Bernoulli}(p_{\text{right}}) \quad (2)$$

Table 5: Parámetros de la distribución Bernoulli para `dominant_side`.

Deporte	$p(\text{right})$	$p(\text{left})$
Fútbol	0.85	0.15
Tenis	0.90	0.10
Baloncesto	0.88	0.12
Balonmano	0.85	0.15
Hockey	0.70	0.30
eSports	1.00	0.00

3.3.3 Variable objetivo: `decision_zone`

La zona de decisión se muestrea de una distribución Multinomial cuyos parámetros son función del contexto:

$$\text{decision_zone} \sim \text{Multinomial}(1, \mathbf{p}(\text{presión}, \text{fatiga}, \text{lado})) \quad (3)$$

Las probabilidades base $\mathbf{p}^{(0)}$ se modifican según reglas de dominio deportivo antes de normalizar. La función `normalizar_probabilidades` garantiza que $\mathbf{p} \geq 0$ y $\sum_k p_k = 1$:

```

1 def normalizar_probabilidades(probs):
2     probs = np.array([max(0.0, p) for p in probs])
3     suma = np.sum(probs)
4     if suma > 0:
5         return probs / suma
6     return np.ones(len(probs)) / len(probs)

```

Listing 1: Normalización de probabilidades.

3.4 Zonas de decisión por deporte

Table 6: Zonas de decisión (`decision_zone`) y acciones por deporte.

Deporte	Acción	Zonas de decisión	<i>K</i>
Fútbol	Penalti	left, center, right	3
Tenis	Saque	wide, body, center	3
Baloncesto	Tiro decisivo	drive, mid_range, three_point	3
Balonmano	Lanzamiento 7m	low_left, low_right, high_left, high_right, center	5
Hockey	Shootout	left_deke, right_deke, direct_shot, high_shot, low_shot	5
eSports	Duelo 1v1	aggressive_push, defensive_hold, flank, bait, direct_duel	5
Total de clases:			24*

*Las zonas son exclusivas por deporte, efectivamente 22 clases en el clasificador.

3.5 Reglas de comportamiento programadas

Cada función generadora aplica modificaciones deterministas sobre las probabilidades base en función del contexto. La Tabla 7 resume estas reglas tal como se extraen del código fuente.

Table 7: Reglas de dominio deportivo aplicadas durante la generación.

Deporte	Condición	Efecto sobre p
Fútbol	Diestro vs zurdo	$p(\text{left})$: +0,05 para diestros
Fútbol	<code>pressure_level == 'alta'</code>	$p(\text{center})+0,15$; laterales $-0,075$ c/u
Tenis	<code>fatigue > 0.7</code>	$p(\text{body})+0,20$; wide, center $-0,10$ c/u
Tenis	<code>pressure_level == 'alta'</code>	$p(\text{body})+0,15$
Baloncesto	<code>fatigue > 0.75</code>	$p(\text{three_point})-0,15$; mid_range +0,20
Baloncesto	<code>pressure_level == 'alta'</code>	$p(\text{drive})+0,10$; three_point $-0,10$
Balonmano	Diestro	$p(\text{low_right})+0,1$; $p(\text{high_right})+0,1$
Balonmano	<code>pressure_level == 'alta'</code>	$p(\text{center})+0,20$; bajos +0,05 c/u
Hockey	<code>fatigue > 0.8</code>	dekes $-0,10$ c/u; direct_shot +0,15; low +0,05
Hockey	<code>pressure_level == 'alta'</code>	$p(\text{direct_shot})+0,20$
eSports	<code>fatigue > 0.8</code>	$p(\text{hold})+0,20$; push $-0,15$
eSports	<code>pressure_level == 'alta'</code>	$p(\text{hold})+0,15$; $p(\text{bait})+0,10$; push $-0,10$

3.6 Semillas de reproducibilidad

Table 8: Semillas NumPy por deporte.

Deporte	Función	Semilla (seed)
Fútbol	generar_penaltis_futbol	42
Tenis	generar_saques_tenis	43
Baloncesto	generar_tiros_baloncesto	44
Balonmano	generar_lanzamientos_balonmano	45
Hockey	generar_shootouts_hockey	46
eSports	generar_duelos_esports	47

4 Índice de presión ponderado

4.1 Definición formal

Definición 4.1 (Índice de presión). *Dado un evento deportivo caracterizado por el tiempo de partido t , la diferencia de marcador d , la importancia de la competición $c \in [0, 1]$, la fatiga del jugador $f \in [0, 1]$ y el momento crítico $m \in [0, 1]$, el índice de presión se define como:*

$$P = 0,30 T(t) + 0,25 S(d) + 0,20 C + 0,15 F + 0,10 M \quad (4)$$

con P acotado a $[0, 1]$.

4.2 Componentes

4.2.1 Presión temporal T

$$T(t) = \frac{t}{t_{\max}}, \quad T \in [0, 1] \quad (5)$$

La función es lineal: cuanto más avanzado el partido, mayor la presión temporal. Para $t < 0$ se devuelve $T = 0$; para $t > 100$ (en el módulo `pressure.py`, donde `match_time` se interpreta como porcentaje) se satura a $T = 1$.

4.2.2 Presión por marcador S

$$S(d) = \frac{1}{1 + |d|} \quad (6)$$

Esta función es una hipérbola decreciente en $|d|$. Su valor máximo $S = 1$ se alcanza en el empate ($d = 0$), y decrece monotónamente a medida que aumenta la ventaja o la desventaja. Algunos valores ilustrativos:

$ d $	0	1	2	3
$S(d)$	1.000	0.500	0.333	0.250

Nota de implementación

La función `calcular_presion_marcador` del módulo `pressure.py` (versión `src/`) utiliza la fórmula $S(d) = 1/(1 + |d|)$. Sin embargo, la versión equivalente del generador sintético usa una aproximación lineal: $S_{\text{gen}}(d) = \max(0, 1 - 0,15 |d|)$. Esta discrepancia entre módulos es una deuda técnica del proyecto.

4.2.3 Importancia competitiva C , fatiga F y momento crítico M

Las tres variables restantes se incorporan directamente como valores continuos en $[0, 1]$:

- C (`competition_importance`): proporcionado como entrada; valores cercanos a 1 corresponden a finales de competición mundial.
- F (`fatigue_level`): proporcionado como entrada; valores altos ($F > 0,7$) activan algunas de las reglas de comportamiento del generador.
- M (`critical_moment`): valor en $[0, 1]$ generado condicionalmente (sección 3.3).

4.3 Pesos y justificación

Table 9: Pesos del índice de presión y su justificación.

Factor	Variable	Peso	Justificación
T	Tiempo	0.30	Factor más determinante en la literatura deportiva
S	Marcador	0.25	Los empates generan la máxima incertidumbre
C	Importancia	0.20	Finales vs. amistosos: diferencia cualitativa
F	Fatiga	0.15	Afecta a la toma de decisiones
M	Momento	0.10	Complemento para situaciones especiales
$\sum = 1,00$			

Los pesos se establecen como primera aproximación basada en criterio experto. El propio marco matemático del proyecto reconoce explícitamente que son “optimizables con datos reales”.

4.4 Clasificación del índice

$$\text{pressure_level} = \begin{cases} \text{baja} & \text{si } P < 0,35 \\ \text{media} & \text{si } 0,35 \leq P < 0,70 \\ \text{alta} & \text{si } P \geq 0,70 \end{cases} \quad (7)$$

4.5 Implementación

```

1 def calcular_pressure_index(
2     match_time, score_difference,
3     competition_importance, fatigue_level, critical_moment=0
4 ):
5     T = calcular_presion_temporal(match_time)
6     S = calcular_presion_marcador(score_difference)
7     C = max(0, min(1, float(competition_importance)))
8     F = max(0, min(1, float(fatigue_level)))

```

```
9     M = max(0, min(1, float(critical_moment)))
10
11     pressure_index = (
12         0.30 * T + 0.25 * S + 0.20 * C +
13         0.15 * F + 0.10 * M
14     )
15     return round(max(0, min(1, pressure_index)), 4)
```

Listing 2: Implementación completa del índice de presión (`src/pressure.py`).

5 Preprocesado de datos

5.1 Pipeline de preprocesado

El módulo `src/preprocessing.py` expone una función principal `preparar_dataframe` que ejecuta los pasos de la Figura 2.

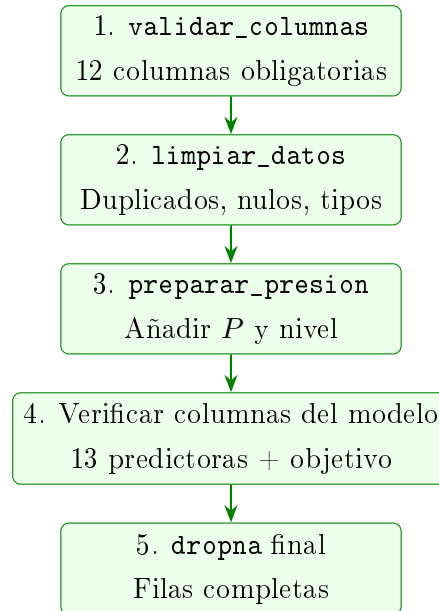


Figure 2: Etapas del preprocesado de datos.

5.2 Columnas obligatorias

El sistema valida la presencia de las siguientes 12 columnas antes de cualquier procesamiento:

```
sport, action_type, competition, match_time, score_difference,  
is_home_player, player, opponent, dominant_side,  
fatigue_level, competition_importance, decision_zone
```

5.3 Limpieza de datos

Table 10: Estrategia de imputación de nulos por variable.

Variable	Tipo	Imputación
match_time	Numérica	Mediana del conjunto
score_difference	Numérica	0 (empate)
is_home_player	Binaria	0
fatigue_level	Numérica	0.5 (punto medio)
competition_importance	Numérica	0.5 (punto medio)
Catégoricas	String	Baja a minúsculas, strip
decision_zone	String	Eliminar fila

5.4 Variables del modelo

Tras el preprocesado, el conjunto de predictoras $\mathbf{x}$ se compone de las 13 variables de la Tabla 11.

Table 11: Variables predictoras del modelo.

Variable	Tipo de dato	Codificación
sport	Catégorica	OneHot
action_type	Catégorica	OneHot
competition	Catégorica	OneHot
player	Catégorica	OneHot
opponent	Catégorica	OneHot
dominant_side	Catégorica	OneHot
pressure_level	Catégorica	OneHot
match_time	Numérica	StandardScaler
score_difference	Numérica	StandardScaler
is_home_player	Numérica	StandardScaler
fatigue_level	Numérica	StandardScaler
competition_importance	Numérica	StandardScaler
pressure_index	Numérica	StandardScaler

6 Modelo de Machine Learning

6.1 Arquitectura del pipeline de sklearn

El modelo se implementa como un Pipeline de scikit-learn con dos etapas:

1. **ColumnTransformer**: aplica `OneHotEncoder` a las 7 variables categóricas y `StandardScaler` a las 6 numéricas.
2. **LogisticRegression**: clasificación multinomial con función de activación softmax.

```

1 def crear_modelo():
2     preprocesado = ColumnTransformer(transformers=[
3         ("cat", OneHotEncoder(handle_unknown="ignore"),
4         columnas_categoricas),
5         ("num", StandardScaler(),
6         columnas_numericas),
7     ])
8     modelo = Pipeline(steps=[
9         ("preprocesado", preprocesado),
10        ("modelo", LogisticRegression(max_iter=5000))
11    ])
12    return modelo

```

Listing 3: Creación del pipeline (src/train_model.py).

6.2 Formulación matemática

6.2.1 Codificación y espacio de características

Sea $\mathbf{x} \in \mathbb{R}^{13}$ el vector original de 13 características. Tras el `ColumnTransformer`:

- Las 7 categóricas generan un vector *one-hot* $\mathbf{x}_c \in \{0, 1\}^{D_c}$ donde D_c depende de la cardinalidad de cada variable.
- Las 6 numéricas se transforman a $\mathbf{x}'_n = (\mathbf{x}_n - \boldsymbol{\mu})/\boldsymbol{\sigma}$.

El vector codificado resultante es $\mathbf{x}' = [\mathbf{x}_c \parallel \mathbf{x}'_n] \in \mathbb{R}^D$.

6.2.2 Regresión logística multinomial (softmax)

Para $K = 22$ clases, el modelo estima:

$$P(y = z_k \mid \mathbf{x}') = \frac{\exp(\mathbf{w}_k^\top \mathbf{x}' + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x}' + b_j)}, \quad k = 1, \dots, K \quad (8)$$

6.2.3 Función de pérdida

El entrenamiento minimiza la entropía cruzada multiclase:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \mathbf{1}[y_i = z_k] \cdot \log P(y_i = z_k \mid \mathbf{x}_i') \quad (9)$$

6.2.4 Decisión

$$\hat{y} = \arg \max_k P(y = z_k \mid \mathbf{x}') \quad (10)$$

La función `predecir_probabilidades` devuelve la distribución completa ordenada de mayor a menor probabilidad, sin colapsar a una única clase.

6.3 División train/test

$$|D_{\text{train}}| = 0,75 \cdot N = 1350, \quad |D_{\text{test}}| = 0,25 \cdot N = 450 \quad (11)$$

El split se realiza con estratificación sobre y cuando todas las clases tienen al menos dos ejemplos:

```

1 conteos = y.value_counts()
2 stratify = y if conteos.min() >= 2 else None
3 return train_test_split(X, y, test_size=0.25,
4                           random_state=42, stratify=stratify)
```

Listing 4: División estratificada con fallback.

6.4 Hiperparámetros

Table 12: Hiperparámetros del clasificador logístico.

Parámetro	Valor	Justificación
<code>max_iter</code>	5000	Asegurar convergencia con 22 clases y alta dimensionalidad OHE
<code>solver</code>	<code>lbfgs</code>	Defecto de <code>sklearn</code> ; eficiente para multiclase
<code>multi_class</code>	<code>auto</code>	Activa softmax multinomial
<code>random_state</code>	42	Reproducibilidad

7 Análisis descriptivo

7.1 Módulo `src/analysis.py`

El módulo implementa cinco funciones de análisis basadas en tablas de contingencia normalizadas por fila (`pd.crosstab(..., normalize="index")`):

Table 13: Funciones de análisis descriptivo.

Función	Tabla obtenida
<code>decisiones_por_deporte(df)</code>	$P(\text{zona} \mid \text{sport})$
<code>decisiones_por_presion(df)</code>	$P(\text{zona} \mid \text{pressure_level})$
<code>decisiones_por_deporte_y_presion(df)</code>	$P(\text{zona} \mid \text{sport}, \text{pressure_level})$
<code>decisiones_por_lado_dominante(df)</code>	$P(\text{zona} \mid \text{dominant_side})$
<code>presion_media_por_deporte(df)</code>	$\mathbb{E}[P \mid \text{sport}]$

7.2 Presión media esperada por deporte

Los valores de presión media esperada se derivan de las distribuciones de generación. La fórmula $\mathbb{E}[P]$ para cada deporte es:

$$\mathbb{E}[P] = 0,30 \mathbb{E}[T] + 0,25 \mathbb{E}[S] + 0,20 \mathbb{E}[C] + 0,15 \mathbb{E}[F] + 0,10 \mathbb{E}[M] \quad (12)$$

Para el hockey sobre hielo, dado que $T = 1$ (siempre final), $d = 0$ (siempre empate, $S = 1$) y $M \sim \mathcal{U}(0,7, 1,0)$, el índice de presión esperado es el más alto del sistema:

$$\mathbb{E}[P_{\text{hockey}}] = 0,30 \cdot 1 + 0,25 \cdot 1 + 0,20 \cdot \mathbb{E}[C] + 0,15 \cdot \mathbb{E}[F] + 0,10 \cdot 0,85 \approx 0,77 \quad (13)$$

8 Interfaz web (Streamlit)

8.1 Arquitectura de app.py

La aplicación utiliza el patrón `@st.cache_data` / `@st.cache_resource` para evitar recargas innecesarias:

- `@st.cache_data`: cachea el CSV ya preprocesado entre recargas de página.
- `@st.cache_resource`: cachea el modelo entrenado; se reentrena solo cuando cambia el dataset.

8.2 Componentes de la interfaz

Table 14: Componentes principales de la aplicación Streamlit.

Componente	Descripción
Sidebar izquierdo	Selector de deporte, tipo de acción y competición
Panel principal — inputs	Sliders para: momento del partido, diferencia de marcador, local/visitante, fatiga, importancia, momento crítico
Gauge de presión	Gráfico <code>plotly.Indicator</code> que muestra $P \in [0, 1]$ en tiempo real
Panel de jugadores	Selectores de jugador y rival con lateralidad automática (mapeada por jugador real)
Predicción	Zona más probable + tabla de probabilidades por clase + gráfico de barras horizontal
Animación	HTML/CSS injected via <code>streamlit.components.v1.html</code> mostrando la trayectoria del balón
Votación del público	Mecanismo de votación antes de revelar la predicción
Tabs de análisis	Dataset completo, distribución de presión, tablas de decisiones, métricas del modelo

8.3 Módulo de animaciones (src/animation.py)

El módulo implementa animaciones HTML/CSS específicas por deporte bajo el design system “Apex Velocity”, generado con Stitch MCP. Cada animación:

1. Renderiza el escenario (portería de fútbol, cancha de tenis, etc.) con CSS `radial-gradient` y `repeating-linear-gradient`.

2. Anima el objeto (balón, raqueta, puck) mediante keyframes CSS (`@keyframes shoot`) con trayectoria paramétrica definida por la zona predicha.
3. Inyecta la animación en Streamlit con `components.html(html, height=420)`.

9 Evaluación y resultados

9.1 Métricas implementadas

La función `evaluar_modelo` calcula:

$$\text{Accuracy} = \frac{\#\text{predicciones correctas}}{N_{\text{test}}} \quad (14)$$

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k} \quad (15)$$

$$\text{Recall}_k = \frac{TP_k}{TP_k + FN_k} \quad (16)$$

$$F_1^{(k)} = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \quad (17)$$

9.2 Resultados con datos sintéticos

Table 15: Resultados de evaluación reportados en el informe técnico del proyecto.

Métrica	Valor	Referencia aleatoria
Accuracy global	$\approx 34\%$	$1/22 \approx 4,5\%$
Mejora sobre azar	$\times 7,5$	—

Interpretación del accuracy

Un 34% sobre 22 clases indica que el modelo captura los patrones programados en los datos sintéticos. La mejora de $\times 7,5$ sobre azar confirma que el sistema discrimina correctamente entre deportes (las zonas de decisión son exclusivas por deporte) y captura los sesgos de presión y fatiga dentro de cada uno.

9.3 Comportamiento por deporte

Table 16: Observaciones principales por deporte según el informe técnico.

Deporte	K	Observación principal
Fútbol	3	Distingue bien left/center/right
Tenis	3	Captura efecto de fatiga en body
Baloncesto	3	Separa drive/mid_range/three_point
Balonmano	5	Detecta influencia de mano dominante
Hockey	5	direct_shot bien predicho en presión alta
eSports	5	defensive_hold bien predicho en presión alta

10 Análisis crítico: limitaciones y trabajo futuro

10.1 Limitaciones identificadas

- L1. Circularidad datos-modelo.** El modelo aprende exactamente los patrones que fueron programados en el generador de datos. Cualquier medida de accuracy es, en consecuencia, artificialmente optimista: el sistema no generaliza a comportamiento deportivo real, sino que recupera sus propias reglas.
- L2. Pesos de presión fijos.** Los coeficientes $\{0,30, 0,25, 0,20, 0,15, 0,10\}$ de la ecuación (4) se establecen por criterio experto sin respaldo empírico. No hay evidencia de que el tiempo tenga exactamente $1,2\times$ más influencia que el marcador.
- L3. Inconsistencia entre módulos.** La función $S(d)$ en `pressure.py` es $1/(1 + |d|)$, mientras que en `synthetic_multisport_data.py` se usa $\max(0, 1 - 0,15 |d|)$. Esto introduce una discrepancia entre los datos generados y la función evaluada en producción.
- L4. Separabilidad lineal.** La regresión logística asume que las clases son linealmente separables en el espacio codificado. Con 22 clases y alta cardinalidad en `player/opponent`, esta hipótesis puede ser insuficiente.
- L5. Sin modelado temporal.** El historial del jugador (rachas, rendimiento reciente, actuaciones contra el rival concreto) no se modela. Cada evento se trata como i.i.d.
- L6. Datos `player/opponent` como predictoras.** Con nombres reales de jugadores en datos sintéticos generados aleatoriamente, estas variables pueden actuar como ruido o introducir *data leakage* implícito en un contexto real.

10.2 Trabajo futuro

Table 17: Extensiones futuras ordenadas por impacto esperado.

#	Extensión	Descripción
1	Datos reales (StatsBomb)	Integrar el open-data de StatsBomb con penaltis reales, incluyendo coordenadas y resultado.
2	Random Forest / XGBoost	Modelos ensemble que capturan no-linealidades y proveen <code>feature_importances_</code> interpretables.
3	SHAP	<i>Waterfall plot</i> por predicción individual: qué variable contribuyó más a la zona predicha.
4	Optimización de pesos	Aprender $\{w_T, w_S, w_C, w_F, w_M\}$ mediante regresión multivariante sobre <code>action_result</code> .
5	Validación cruzada k -fold	Sustituir el split 75/25 por k -fold estratificado para resultados más robustos.
6	Series temporales del jugador	Añadir racha reciente, historial vs. rival, y presión acumulada como variables derivadas.
7	Dockerización	Empaquetado de la app Streamlit para despliegue reproducible.

11 Suite de tests

El archivo `test_synthetic_multisport_data.py` implementa una suite de tests unitarios sobre el generador de datos. Los tests verifican:

- Que los CSV se generan y contienen las columnas esperadas.
- Que los rangos de las variables numéricas son correctos ($[0, 1]$ para fatiga, $[0, 1]$ para `pressure_index`, etc.).
- Que los deportes generados coinciden con los seis esperados.
- Que las zonas de decisión de cada deporte son exactamente las definidas.
- Que el número de filas por deporte es $N = 300$.

Se ejecutan con:

```
1 python -m pytest test_synthetic_multisport_data.py -v
```

12 Evolución del proyecto: de `Mates.py` al framework multideporte

12.1 Versión original: `Mates.py`

El archivo `Mates.py` es el punto de partida histórico del proyecto. Implementa el mismo índice de presión pero restringido a fútbol (penaltis), con una sola zona de salida y sin el pipeline de `sklearn`. Constituye el prototipo que demostró la viabilidad del concepto antes de generalizarlo.

12.2 Generalización multideporte

La transición de `Mates.py` al framework actual implicó:

1. Abstraer el esquema de columnas en un conjunto único (`COLUMNAS_OBLIGATORIAS`, `COLUMNAS_MODELO`).
2. Parametrizar las funciones de generación por deporte.
3. Unificar el preprocesado en un `ColumnTransformer` agnóstico al deporte.
4. Separar la lógica de animación en un módulo propio (`animation.py`) con un design system compartido.
5. Generar una versión corregida (`app_corregida/`) que resuelve bugs de la versión raíz.

13 Conclusiones

Este proyecto constituye un **framework completo y reproducible** para la predicción de decisiones tácticas en deportes bajo presión. Sus contribuciones principales son:

- C1. Índice de presión multifactor.** La combinación ponderada $P = 0,30T + 0,25S + 0,20C + 0,15F + 0,10M$ encapsula cinco dimensiones del contexto situacional en un único escalar interpretable, clasificado en tres niveles que modulan directamente la variable objetivo.
- C2. Arquitectura modular.** La separación en módulos independientes (`pressure.py`, `preprocessing.py`, `train_model.py`, `analysis.py`, `animation.py`) permite sustituir cualquier componente sin afectar al resto. En particular, la migración de datos sintéticos a datos reales (StatsBomb, Wyscout) no requiere modificar la arquitectura.
- C3. Generalización multideporte.** El mismo pipeline opera sobre seis deportes con zonas de decisión y reglas de dominio propias, demostrando que el marco matemático es suficientemente general.
- C4. Demo interactiva.** La interfaz Streamlit materializa el modelo en un producto funcional que permite explorar escenarios en tiempo real, comparar la predicción del modelo con la intuición humana y visualizar animaciones de la trayectoria predicha.
- C5. Limitación reconocida.** El proyecto identifica explícitamente la circularidad datos-modelo como su limitación principal. El trabajo futuro de mayor impacto es la integración de datos reales, preferiblemente del repositorio open-data de StatsBomb para el deporte del fútbol, y de fuentes equivalentes para los demás.

References

- [1] M. López de Prado, *Advances in Financial Machine Learning*. Wiley, 2018.
- [2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006, cap. 4.
- [4] StatsBomb, *StatsBomb Open Data*, <https://github.com/statsbomb/open-data>, 2023.
- [5] A. Treuille, T. Teixeira, y A. Kelleher, “Streamlit: The fastest way to build and share data apps,” <https://streamlit.io>, 2023.

A Ejemplo de evento sintético completo

Table 18: Ejemplo de fila del dataset para un penalti de fútbol.

Campo	Valor
event_id	FTB_0001
sport	football
action_type	penalty
competition	Champions League
match_time	87 (min)
score_difference	0 (empate)
is_home_player	1
player	Mbappé
opponent	Courtois
dominant_side	right
fatigue_level	0.73
competition_importance	0.92
pressure_index	0.741
pressure_level	alta
decision_zone	center
action_result	goal

Cálculo del índice de presión para este ejemplo:

$$T = 87/120 = 0,725$$

$$S = 1/(1 + 0) = 1,000$$

$$C = 0,92$$

$$F = 0,73$$

$$M \approx 0,82 \quad (\text{alta presión contextual: } t_{\text{norm}} > 0,8, |d| \leq 1)$$

$$\begin{aligned} P &= 0,30 \times 0,725 + 0,25 \times 1,000 + 0,20 \times 0,92 + 0,15 \times 0,73 + 0,10 \times 0,82 \\ &= 0,218 + 0,250 + 0,184 + 0,110 + 0,082 = \mathbf{0,844} \end{aligned}$$

Con $P = 0,844 \geq 0,70$, el nivel es **alta**, lo que activa la regla: $p(\text{center}) + 0,15$, explicando la decisión de tirar al centro.

B Ejecución del sistema


```
1 # 1. Instalar dependencias
2 pip install -r requirements.txt
3
4 # 2. Regenerar datos sintéticos
5 python synthetic_multisport_data.py
6
7 # 3. Pipeline completo por consola
8 python main.py
9
10 # 4. Interfaz web interactiva
11 streamlit run app.py
12
13 # 5. Suite de tests
14 python -m pytest test_synthetic_multisport_data.py -v
```

Listing 5: Comandos de ejecución.